EFREI Paris — Cycle ingénieur ING-APP1-BDML
Algorithmique et théorie des graphes — Projet intégrateur

Novapolis Résilience

Planification des secours et conception d'un réseau urbain résilient

Rapport de projet

Code source : `https://github.com/AjdahimHassan/TP_GRAPHES`

Hassan AJDAHIM (membre 1)
Andreja JOVANOVIC (membre 2)
Angelo CUNHA VELHO MACHADO (membre 3)

Données synthétiques. Novapolis est une ville fictive. Toutes les données utilisées dans ce rapport ont été produites par notre propre générateur, avec des graines fixées. N'importe quel tableau ou figure peut donc être refait à l'identique avec la commande indiquée en annexe B.

Novembre 2026

Synthèse

La ville de Novapolis voulait un outil capable de répondre à trois questions, et surtout un outil dont on puisse refaire les calculs. Pour ça, nous avons représenté la ville par deux graphes construits sur les mêmes 150 sites : le réseau routier, orienté parce qu'une rue peut être à sens unique, où chaque route porte un temps de trajet, et le réseau de fibre, sans sens, où chaque tronçon a un coût de construction. Tous les algorithmes ont été codés par nous en Python. On a seulement utilisé `networkx` dans les tests, pour vérifier qu'on trouvait les mêmes résultats.

Question de la ville	**Ce qu'on a trouvé (jeu réaliste)**
1. Quels quartiers sont mal desservis ?	149 sites sur 150 sont atteignables depuis les casernes. Le quartier `Q-123` est complètement isolé, et 9 quartiers sont à plus de 10 minutes de toute caserne ou de tout hôpital.
2. Quels itinéraires si des routes ferment ?	Quand on ferme les 3 ponts du centre, personne ne devient inaccessible, mais 4 interventions sont retardées, la pire de 6,56 minutes pour `D-02`. Les trois algorithmes de plus court chemin trouvent les mêmes distances.
3. Quel réseau de fibre, et comment le réparer ?	Le réseau le moins cher coûte 11 130 k€ (147 tronçons), avec Kruskal comme avec Prim. Si `Q-014`–`Q-019` tombe en panne, on le remplace par `Q-014`–`Q-025` (80 k€), soit 15 k€ de plus au total.

Sommaire

1 Le problème et notre démarche . . . 2
2 Modélisation, données et architecture . . . 2
3 Module 1 — Qui peut être atteint ? (DFS, BFS) . . . 3
4 Module 2 — Itinéraires des secours . . . 4
5 Module 3 — Les circuits absorbants . . . 5
6 Module 4 — Le réseau de fibre . . . 6
7 Module 5 — Résilience et scénarios . . . 7
8 Tests et mesures de performance . . . 8
9 Limites, recommandations et démonstration . . . 9
A Contributions de l'équipe . . . 10
B Document joint et commandes . . . 10

1 Le problème et notre démarche

Novapolis, c'est une ville avec des hôpitaux, des casernes de pompiers, des écoles, des centres de télécommunication et beaucoup de quartiers. Après plusieurs incidents, la mairie se demande quels quartiers sont trop loin des secours, quel chemin les pompiers doivent prendre quand des routes sont fermées, et comment construire un réseau de fibre pas trop cher qui puisse être réparé facilement si un tronçon casse.

Le sujet le dit clairement : le but n'est pas de faire une jolie carte, mais d'avoir des résultats justes et de pouvoir le prouver. On a donc suivi trois règles. Tout doit être **vérifiable** : chaque algorithme a été testé sur un petit graphe calculé à la main. Les scénarios doivent être **comparables** : chaque perturbation est appliquée à une copie de la ville, avec les mêmes indicateurs qu'en situation normale. Et tout doit être **reproductible** : les données sortent d'un générateur à graine fixe, et chaque résultat est accompagné de la commande qui permet de le refaire (annexe B). L'outil s'utilise en ligne de commande, par une API ou par une interface web. Les trois appellent le même code Python, et l'interface ne fait aucun calcul elle-même, donc elle affiche forcément les mêmes chiffres que le terminal.

2 Modélisation, données et architecture

2.1 Deux graphes pour une même ville

Pour les routes, on utilise un graphe orienté : une flèche de A vers B veut dire qu'on peut rouler de A vers B. Une rue à double sens donne donc deux flèches, un sens unique une seule. Le poids, c'est le temps de trajet en minutes. Pour la fibre, le sens n'a pas d'importance (un câble relie deux points), donc le graphe est non orienté et le poids est le coût en k€.

Il a aussi fallu décider quel temps donner aux algorithmes. On prend le temps normal ou le temps de pointe selon le scénario. Une route fermée est retirée du graphe, et une route en travaux garde sa place mais son temps est multiplié par 1,5 (c'est notre choix, le sujet ne donnait pas de valeur). Côté fibre, les tronçons interdits ou en panne ne sont jamais proposés. La caserne qui intervient est celle qui arrive le plus vite : au lieu de faire un calcul par caserne, on lance un seul Dijkstra qui part de toutes les casernes en même temps. Enfin, un temps négatif n'a pas de sens dans une vraie ville, donc le validateur les refuse partout sauf dans les deux petits jeux de test du module 3.

2.2 Les fichiers, le validateur et les jeux de données

Un jeu de données, c'est un dossier avec les cinq fichiers CSV du sujet (sites, routes, fibre, demandes, scénarios) et un `meta.json` qui garde la graine, les paramètres du générateur et une empreinte de chaque fichier. Le format complet est décrit dans `data/SCHEMA.md`. Avant tout calcul, le validateur relit les fichiers et, pour chaque problème, donne le fichier, la ligne, la colonne et la valeur fautive. Par exemple, quand on a ajouté exprès deux erreurs au petit jeu, il a répondu « `sites.csv`, ligne 10 : identifiant dupliqué C1 » et « `routes.csv`, ligne 16 : site inexistant ZZZ ». Il repère aussi les colonnes manquantes, les statuts inconnus, les valeurs non numériques et les priorités hors de 1 à 3.

On a préparé plusieurs jeux. Le jeu `mini` (8 sites) se calcule entièrement à la main, et ses résultats attendus sont dans `data/mini/EXPECTED.md`. Deux jeux de 6 sites servent aux tests du module 3. Le jeu réaliste `novapolis` (graine 42) compte 150 sites (6 casernes, 4 hôpitaux, 12 écoles, 5 centres télécom, 123 quartiers), 739 routes orientées, 328 tronçons de fibre, une rivière traversée par 6 ponts, 40 demandes et 6 scénarios. Enfin, 9 gros jeux de 1 000 à 5 000 sites servent aux mesures de performance. Avec la même graine, le générateur redonne exactement les mêmes fichiers, octet par octet.

2.3 Structures de données et organisation du code

Pour chaque algorithme, on a pris la structure qui donne la complexité vue en cours. Les parcours, Dijkstra, Prim et Tarjan utilisent des **listes d'adjacence** : on peut parcourir tout le graphe en un temps proportionnel au nombre de sites plus le nombre de routes, avec peu de mémoire. Kruskal et

Bellman–Ford travaillent sur une simple **liste d'arêtes**, que l'un trie et que l'autre repasse à chaque tour. Floyd–Warshall remplit une **matrice**, qui grossit avec le carré du nombre de sites, d'où son usage limité aux petits graphes. Dijkstra et Prim utilisent un **tas** (`heapq`), le BFS une **file** (`deque`), et Kruskal un **Union–Find**. Tous les parcours sont écrits sans récursion, ce qui évite à Python de planter sur les gros graphes.

Le code est découpé en blocs qui ont chacun un seul rôle (figure 1). Les algorithmes reçoivent un graphe et renvoient un résultat, sans rien savoir de l'affichage. Une couche `services` propose une fonction par opération, et c'est elle qu'appellent la ligne de commande et l'API.

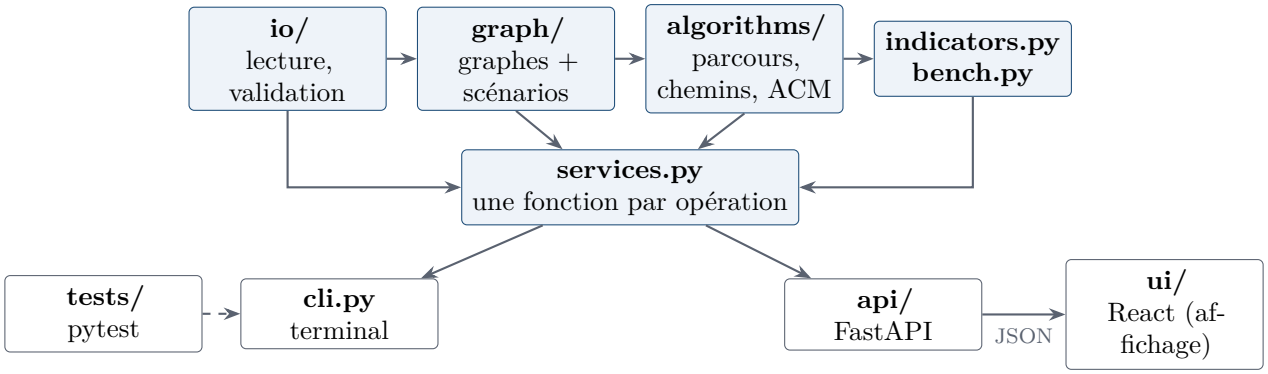


Figure 1 – Organisation du code. Les algorithmes ne dépendent ni de l'affichage ni du format des fichiers.

3 Module 1 — Qui peut être atteint ? (DFS, BFS)

Dans un graphe orienté, « connecté » peut vouloir dire plusieurs choses, et il faut bien les séparer. Pour les secours, la notion qui compte est l'**accessibilité** : existe-t-il un chemin, en respectant le sens des rues, entre une caserne et un site ? On la calcule avec un BFS qui part de toutes les casernes en même temps. La **connexité faible** ignore le sens des rues et sert juste à voir si des morceaux de ville sont complètement coupés. La **connexité forte** demande qu'on puisse aller de A à B *et* revenir. On la calcule avec Tarjan, et elle repère les endroits où l'on peut entrer sans pouvoir ressortir. Pour la fibre, qui n'a pas de sens, on compte simplement les composantes connexes.

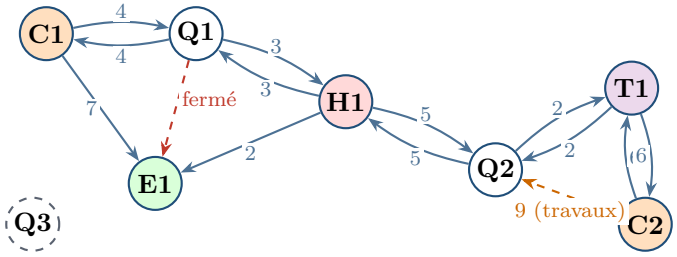


Figure 2 – Les routes du jeu `mini`. En orange, une route en travaux ; en rouge, une route fermée, donc retirée. Q3 n'a aucune route.

Vérification à la main. Le jeu `mini` (figure 2) contient tous les cas piégeux : un sens unique, une route fermée, une route en travaux, un site dont on ne peut pas ressortir (E1) et un site isolé (Q3). Le BFS depuis les deux casernes visite C1, C2, Q1, E1, T1, Q2 puis H1, et n'atteint jamais Q3. Le chemin de C2 à H1 qui prend le moins de routes est C2, Q2, H1 (2 routes, 14 minutes), mais le plus rapide passe par T1 (3 routes, 13 minutes) : « le moins d'étapes » et « le plus rapide », ce n'est pas pareil. Le DFS trouve le circuit C1, Q1, C1 en tombant sur un site encore en cours de visite. On compte enfin 3 composantes fortement connexes mais seulement 2 faiblement connexes, puisque E1 reste relié au reste si on oublie le sens des rues.

Sur la ville complète, 149 sites sur 150 sont atteignables depuis les 6 casernes. Le seul site manquant est `Q-123`, qui n'a aucune route : on l'a mis exprès pour tester ce cas. Le graphe a 2 composantes

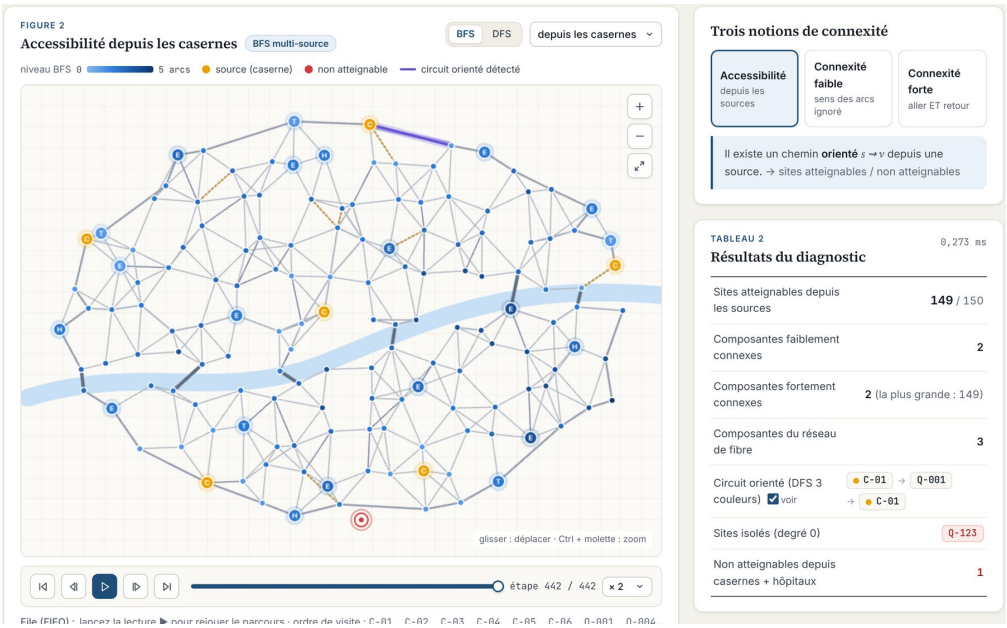


Figure 3 – Le module 1 dans l'interface. La couleur indique à combien de routes chaque site se trouve des casernes (plus c'est foncé, plus c'est loin).

faiblement connexes et 2 fortement connexes (la grande fait 149 sites), et le réseau de fibre en a 3. Le circuit trouvé par le DFS (C-01, Q-001, C-01) n'est qu'une rue à double sens, mais il montre que la détection marche. Comme « être atteignable » ne suffit pas à la mairie, on a ajouté un seuil de 10 minutes : 8 quartiers le dépassent en plus de `Q-123`, le pire étant `Q-040` à 11,9 minutes.

4 Module 2 — Itinéraires des secours

On a codé les trois algorithmes de plus court chemin du cours. **Dijkstra** est utilisé par défaut, parce qu'il est le plus rapide quand tous les temps sont positifs. Il refuse d'ailleurs de se lancer s'il voit un poids négatif, puisqu'il pourrait alors se tromper. **Bellman–Ford** accepte les poids négatifs : il repasse sur toutes les routes, tour après tour, et on l'arrête dès qu'un tour ne change plus rien. **Floyd–Warshall** calcule d'un coup toutes les distances entre toutes les paires de sites, ce qui coûte beaucoup plus cher. Pour chacun, on garde de quoi reconstruire le chemin complet. Sur le jeu `mini`, les trois donnent le même tableau de distances, et sur 50 graphes aléatoires, les mêmes distances que `networkx`.

Scénario : on ferme les trois ponts du centre. On ferme les trois ponts sur la Nova dans les deux sens, puis on recalcule tous les itinéraires sur une copie de la ville.

Demande	Type	Site	Avant	Après	Retard	En %	Caserne
`D-02`	Incendie (P2)	`Q-085`	7,33	13,89	+6,56	+89,5 %	C-04 → C-06
`D-03`	Fuite de gaz (P1)	`Q-080`	9,75	13,46	+3,71	+38,1 %	C-04 → C-05
`D-04`	Effondrement (P1)	`Q-093`	9,60	11,62	+2,02	+21,0 %	C-04 → C-06
`D-05`	Fuite de gaz (P2)	`Q-098`	10,52	11,49	+0,97	+9,2 %	C-04 → C-06

Table 1 – Les demandes retardées (temps en minutes). Les 35 autres ne changent pas, et `D-01`, sur `Q-123`, reste inaccessible.

Pour `D-02`, les trois algorithmes trouvent le même trajet de 13,89 minutes, mais pas dans le même temps : 0,13 ms pour Dijkstra, 0,24 ms pour Bellman–Ford, et 49,5 ms pour Floyd–Warshall, qui calcule toutes les paires alors qu'on n'en utilise qu'une. Toutes les demandes retardées étaient servies par `C-04`, la caserne du centre, qui ne peut plus traverser. En moyenne le retard paraît minuscule (0,34 minute sur les 39 demandes accessibles), mais le pire cas est de 6,56 minutes, ce qui est énorme pour

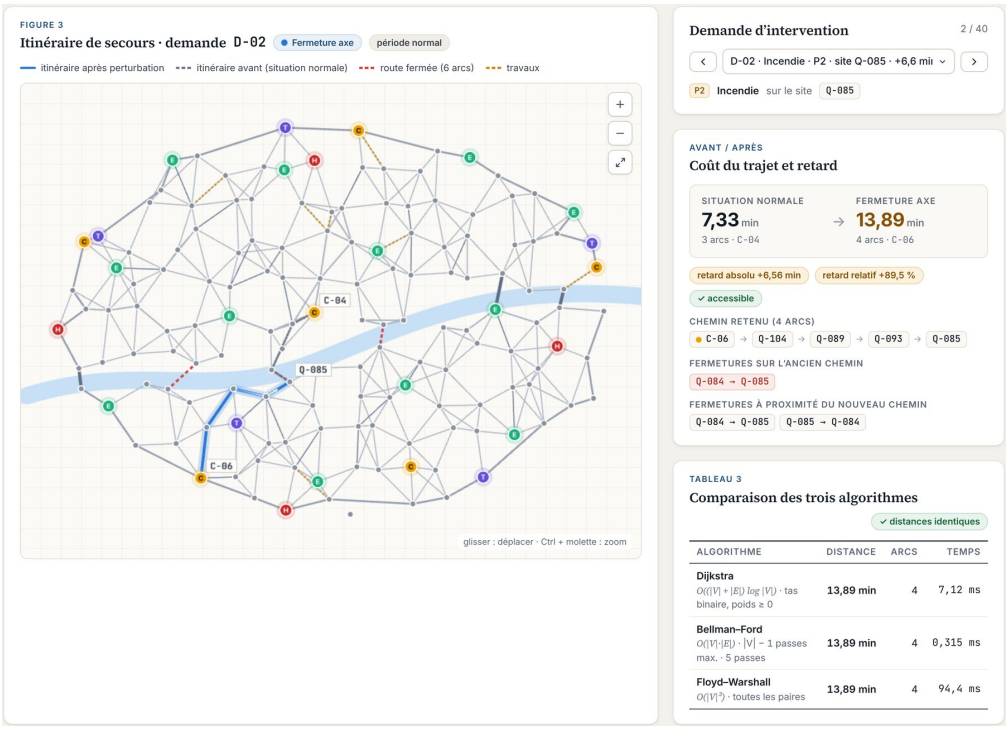


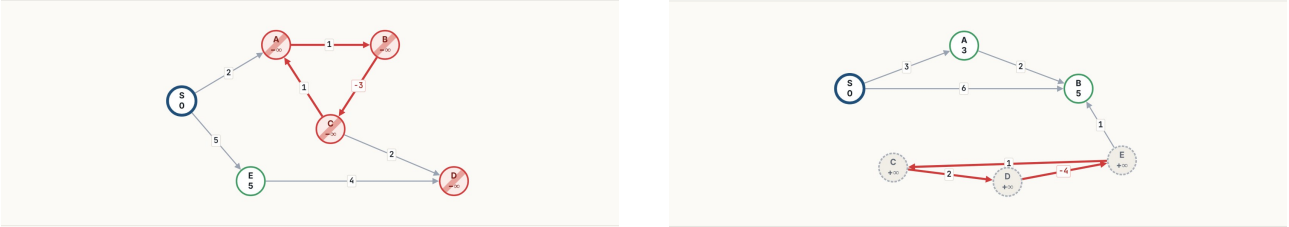
Figure 4 – La demande `D-02` (un incendie) avec les ponts fermés. Avant, `C-04` passait par le pont `Q-084`–`Q-085`. Maintenant c'est `C-06` qui intervient, sans traverser, mais en 13,89 minutes au lieu de 7,33.

un incendie. C'est pour ça qu'on donne toujours la moyenne *et* le maximum.

Limites du modèle. Un plus court chemin, ce n'est pas une vraie intervention. Nos temps sont fixes : pas de trafic en direct, pas de feux, pas de priorité pour les secours. On traite une intervention à la fois, sans savoir si la caserne est déjà occupée. Le facteur 1,5 des travaux et les temps de pointe sont des hypothèses, et le temps calculé ne compte ni le départ ni l'accès au bâtiment.

5 Module 3 — Les circuits absorbants

Un circuit absorbant est une boucle de poids total négatif : à chaque tour, la distance baisse, sans fin, et elle vaut donc $-\infty$. Ça ne peut pas arriver dans une vraie ville, donc on a créé deux petits graphes de test avec des poids négatifs marqués comme artificiels. Avec **Bellman–Ford**, après le nombre normal de tours, on en refait un. Si une distance peut encore baisser, il y a un circuit absorbant atteignable depuis la source, et on met $-\infty$ pour tous les sites qu'on peut atteindre à partir de lui. Avec **Floyd–Warshall**, si la distance d'un site vers lui-même devient négative, ce site est sur un circuit absorbant, et toutes les distances qui peuvent passer par lui valent $-\infty$.



(a) Test A : le circuit est atteignable depuis S.

(b) Test B : le circuit n'est pas atteignable.

Figure 5 – Les deux graphes de test du module 3 dans l'interface (en rouge, les sites à $-\infty$).

Dans le test A, D vaut aussi $-\infty$ alors qu'il n'est pas sur le circuit : on peut l'atteindre en sortant du circuit. E garde une distance normale (5), car aucun chemin ne va du circuit vers lui. Dans le test B, le circuit existe et les deux algorithmes le trouvent, mais S ne peut pas l'atteindre, donc ses distances

Site	Test A : circuit A, B, C atteignable		Test B : circuit C, D, E non atteignable	
	Bellman–Ford	Floyd–Warshall	Bellman–Ford	Floyd–Warshall
S	0	0	0	0
A	$-\infty$	$-\infty$	3	3
B	$-\infty$	$-\infty$	5	5
C	$-\infty$	$-\infty$	$+\infty$	$+\infty$
D	$-\infty$	$-\infty$	$+\infty$	$+\infty$
E	5	5	$+\infty$	$+\infty$

Table 2 – Distances depuis S dans les deux tests (`novapolis negcycle -algo all`). Les deux circuits ont un poids total de -1.

restent normales ou infinies. C'est pour ça que l'outil donne deux informations séparées : « il y a un circuit absorbant » et « il est atteignable depuis la source ».

6 Module 4 — Le réseau de fibre

On cherche à relier tous les sites en payant le moins possible. **Kruskal** trie les tronçons du moins cher au plus cher et les ajoute un par un, sauf ceux qui formeraient une boucle. Pour le savoir vite, il utilise un **Union–Find**, codé avec compression de chemin et union par rang. **Prim** part d'un site et ajoute toujours le tronçon le moins cher qui touche l'arbre. Quand le graphe n'est pas connexe, il recommence depuis un site pas encore relié, ce qui donne une **forêt** couvrante.

Sur le petit jeu, Kruskal trouve un réseau à 55 k€. Prim trouve le même coût, mais garde C2–Q2 au lieu de T1–C2 : les deux coûtent 9, et C2–Q2 est juste arrivé en premier dans sa file. Ce n'est pas une erreur. Si on ajoute à l'arbre un tronçon qui n'en fait pas partie, ça crée une boucle, et si un autre tronçon de cette boucle a exactement le même prix, on peut les échanger. L'arbre minimum n'est unique que si tous les coûts sont différents, ce qui n'est pas notre cas puisqu'ils sont arrondis à 5 k€.

Sur la ville, Kruskal et Prim trouvent tous les deux 11 130 k€ avec 147 tronçons, soit 150 sites moins 3 morceaux séparés, car `Q-058` et `Q-123` n'ont aucun tronçon possible. 4 tronçons diffèrent entre les deux, toujours à cause d'égalités : par exemple `Q-071`–`Q-082` et `Q-082`–`Q-088` coûtent tous les deux 75 k€.

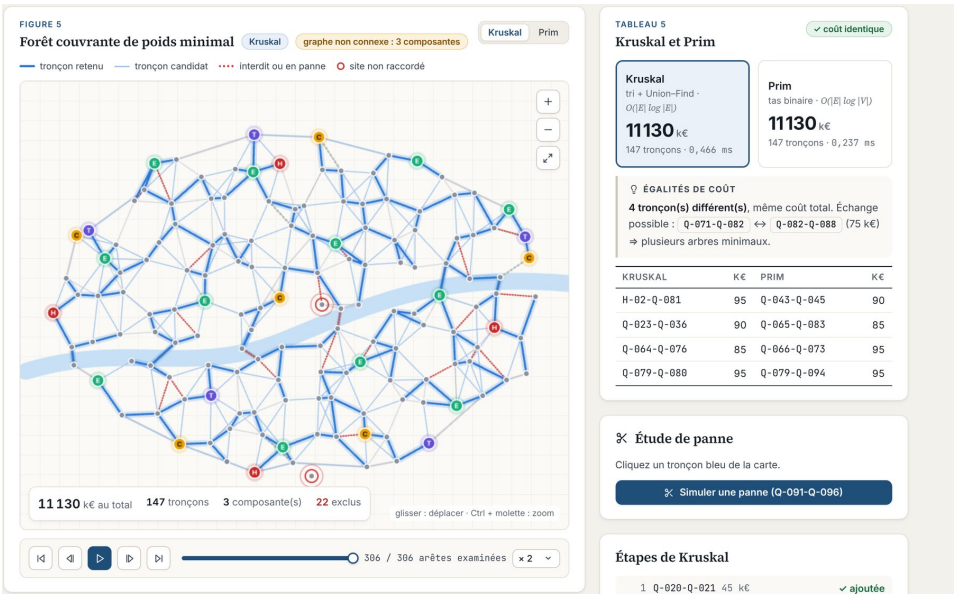
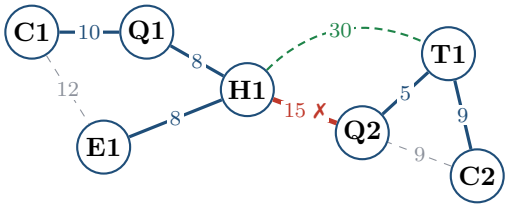


Figure 6 – Le réseau de fibre le moins cher (en bleu), en 3 morceaux. À droite, la comparaison entre Kruskal et Prim.

En cas de panne, l'arbre se coupe en deux. Pas besoin de tout recalculer : il suffit de trouver le

tronçon disponible le moins cher qui relie les deux morceaux (figure 7).



Panne de H1–Q2 : deux morceaux {C1, Q1, H1, E1} et {Q2, T1, C2}. Seul lien possible : H1–T1 (30 k€), d'où $55 - 15 + 30 = 70$ k€.

Figure 7 – Panne sur le petit jeu, calculée à la main puis confirmée par l'outil.

Sur la ville, on a testé la panne de `Q-014`–`Q-019` (65 k€), qui coupe le réseau en deux morceaux de 77 et 71 sites. Le meilleur raccordement est `Q-014`–`Q-025`, à 80 k€. Le coût total passe de 11 130 à 11 145 k€, seulement 15 k€ de plus, mais la ville doit quand même payer 80 k€ pour construire le nouveau tronçon. C'est ce coût-là qu'on reprend dans le module 5.

7 Module 5 — Résilience et scénarios

Le sujet demandait au moins trois scénarios, on en a fait six : les trois imposés (fermeture des ponts du centre, panne du tronçon `Q-014`–`Q-019`, zone de travaux de 25 routes autour de `C-04`), plus l'heure de pointe sur toutes les routes et une crue qui coupe les sites `E-07` et `Q-052` au bord de la rivière. On ne modifie jamais la ville de départ : chaque scénario travaille sur une copie, et une empreinte (SHA-256) des données, calculée avant et après chaque calcul, prouve qu'elles n'ont pas changé.

Indicateur	normal	ponts	panne fibre	pointe	travaux	crue
Demandes devenues inaccessibles	0	0	0	0	0	**1**
Hausse moyenne du trajet (min)	0,00	0,34	0,00	**3,55**	0,31	0,00
Hausse maximale du trajet (min)	0,00	6,56	0,00	**8,26**	2,28	0,00
Composantes fibre (avant réparation)	3	3	**4**	3	3	3
Coût du raccordement (k€)	0	0	**80**	0	0	0
Sites sensibles accessibles	100 %	100 %	100 %	100 %	100 %	**96,3 %**

Table 3 – Les cinq indicateurs pour chaque scénario (`novapolis scenarios -all`). En gras, la pire valeur de chaque ligne.

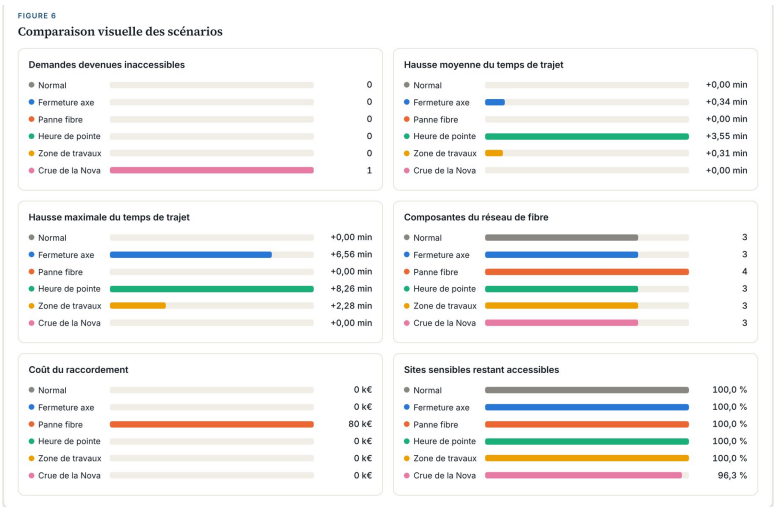


Figure 8 – Un petit graphique par indicateur, une barre par scénario.

La première chose qu'on remarque, c'est que les scénarios n'abîment pas la même chose. Les ponts et la pointe rallongent les trajets, la panne touche la fibre, et la crue rend des sites inaccessibles. Avec un seul indicateur, on serait passé à côté de deux scénarios sur trois. L'heure de pointe coûte le plus de

temps au total : elle touche 39 demandes sur 40, avec 3,55 minutes de plus en moyenne et jusqu'à 8,26 pour `D-27`. Les ponts, eux, ne touchent que 4 demandes, mais beaucoup, et les travaux restent locaux (11 demandes, toutes servies par `C-04`, au plus 2,28 minutes).

La crue est un cas particulier. Elle ne change aucun temps, mais l'incendie `D-29` sur `Q-052`, une priorité 1, devient impossible à atteindre, et l'école `E-07` est coupée. La hausse moyenne reste à zéro parce qu'on ne peut pas calculer de retard pour une demande qu'on n'atteint plus. Il ne faut donc jamais lire cet indicateur seul, mais toujours avec le nombre de demandes devenues inaccessibles. Enfin, la panne de fibre fait passer le réseau de 3 à 4 morceaux, et une réparation à 80 k€ le ramène à 3.

8 Tests et mesures de performance

Cette partie est facultative dans le sujet, mais elle nous a servi à vérifier notre travail. Nous avons écrit **151 tests `pytest`**, qui passent tous en 3 secondes. Ils vérifient tous les résultats calculés à la main sur le petit jeu, comparent nos résultats à ceux de `networkx` sur 100 graphes aléatoires, testent les cas limites (graphe vide, départ égal à l'arrivée, toutes les routes fermées, chaîne de 20 000 sites), les deux tests du module 3, chaque erreur du validateur, et le fait que la ville de base n'est jamais modifiée.

Pour les mesures, tous les algorithmes tournent sur les mêmes graphes : 5 tailles de 250 à 5 000 sites, 3 densités, graine 42. On ne chronomètre que l'algorithme, chaque mesure est refaite 5 fois, et on garde la moyenne. Floyd–Warshall est mesuré à part, jusqu'à 400 sites, car il devient vite trop lent.

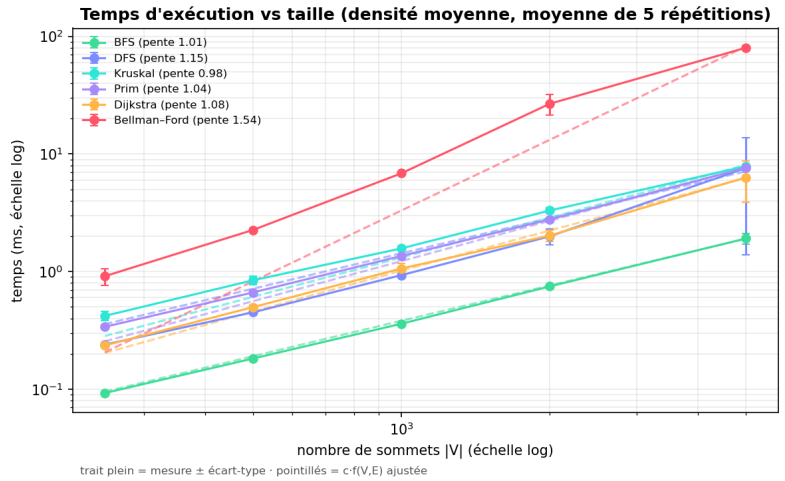
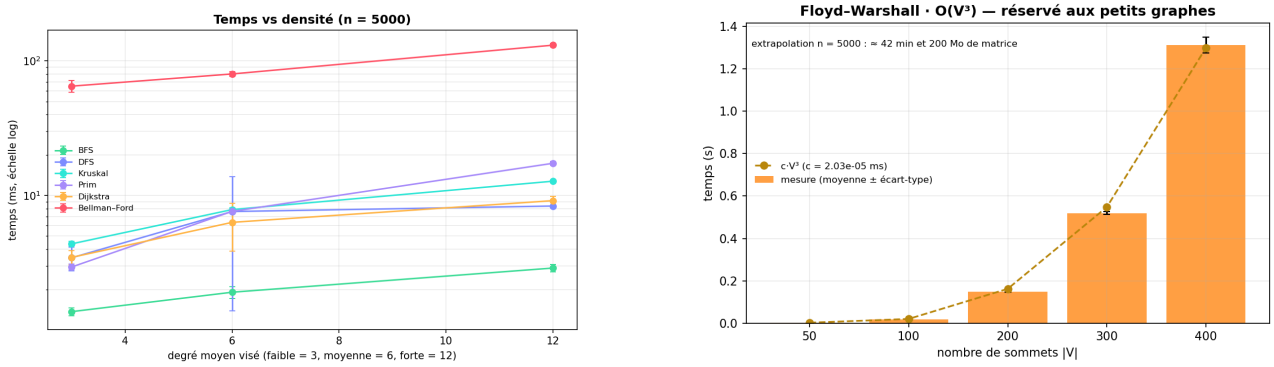


Figure 9 – Temps de calcul selon le nombre de sites (échelles logarithmiques, densité moyenne). En pointillés, la courbe que prévoit la théorie.



(a) Temps selon la densité, à taille fixée.

(b) Floyd–Warshall, jusqu'à 400 sites.

Figure 10 – Effet de la densité et coût de Floyd–Warshall.

Sur une échelle logarithmique, un temps qui grandit comme n^p donne une droite de pente p. Pour BFS,

	BFS	DFS	Kruskal	Prim	Dijkstra	Bellman–Ford	Floyd–Warshall
Pente mesurée	1,01	1,15	0,98	1,04	1,08	1,54	2,91
Pente théorique	1	1	≈1,1	≈1,1	≈1,1	2	3

Table 4 – Pentes mesurées (densité moyenne) comparées à la théorie.

DFS, Dijkstra, Kruskal et Prim, on mesure des pentes entre 1,0 et 1,15, ce qui colle à la théorie. Le DFS est quand même 2 à 4 fois plus lent que le BFS, parce que notre version sans récursion gère une pile un peu lourde, ce qui ne se voit pas dans une notation en O. Floyd–Warshall a une pente de 2,91, très proche du 3 attendu : il met déjà 1,3 seconde pour 400 sites, et il faudrait environ 43 minutes pour 5 000.

Le résultat le plus intéressant est celui de **Bellman–Ford** : la théorie prévoit une pente de 2, mais on mesure environ 1,5. Ça vient de notre arrêt anticipé. Le nombre de tours nécessaires correspond en gros à la longueur du plus long des plus courts chemins, et pas au nombre de sites : 23 tours pour 250 sites et 96 pour 5 000, très loin des 4 999 du pire cas. Dans un graphe qui ressemble à une ville, cette longueur grandit à peu près comme la racine carrée du nombre de sites, d'où un coût en $n^{1,5}$. La complexité théorique reste juste, mais c'est un pire cas qu'on n'atteint pas ici.

9 Limites, recommandations et démonstration

Notre travail a des limites. Les données sont inventées : la méthode fonctionne, mais les chiffres ne décrivent pas une vraie ville, et il faudrait utiliser des données ouvertes comme OpenStreetMap pour l'appliquer. Le modèle est statique, avec une intervention à la fois. Les coûts de fibre sont supposés, et on n'a testé que quelques scénarios choisis. La suite logique serait de retirer chaque pont et chaque tronçon un par un pour trouver automatiquement les points les plus fragiles.

À partir de nos résultats, voici ce qu'on recommanderait à la ville :

1. **Désenclaver `Q-123` et raccorder `Q-058` à la fibre**, les deux seuls sites hors réseau.
2. **Sécuriser les ponts du centre**, par exemple avec une caserne relais sur la rive sud près de `Q-085`, puisque leur fermeture retarde 4 interventions.
3. **Prévoir à l'avance le tronçon `Q-014`–`Q-025`** (80 k€) pour réparer tout de suite une panne de `Q-014`–`Q-019`.
4. **Surveiller les 8 quartiers entre 10 et 12 minutes**, comme `Q-040`, qui seront les premiers hors délai à l'heure de pointe.
5. **Protéger `Q-052` et `E-07` contre les crues**, seul scénario où l'on perd une intervention de priorité 1.

Pour la démonstration de 10 minutes, l'interface a un mode « Présenter » qui suit l'ordre du sujet, du validateur jusqu'aux scénarios. Si le jury pose une question précise, la page « Algorithmes en direct » permet de relancer n'importe quel algorithme sur le graphe et le départ de son choix.

A Contributions de l'équipe

Membre	Modules principaux	Fichiers et livrables
Hassan AJDAHIM (membre 1)	Développement : modules M1 à M5, données, CLI, API et interface	`novapolis/`, `tests/`, `scripts/`, `data/`, `ui/` ; rapport : modélisation, architecture
Andreja JOVANOVIC (membre 2)	Théorie M1, M2, M3 : parcours, connexité, plus courts chemins, circuits absorbants	Vérification à la main de M1 à M3 sur le jeu `mini` ; rapport : problème, M1, M2, M3
Angelo CUNHA VELHO MACHADO (membre 3)	Théorie M4, M5, benchmarks : arbres couvrants, résilience, analyse des mesures	Vérification à la main de M4 et M5 sur le jeu `mini`, analyse des benchmarks ; rapport : M4, M5, expériences, limites

Table 5 – Qui a fait quoi (le détail est dans `CONTRIBUTIONS.md`).

B Document joint et commandes

En plus de ce rapport, nous joignons un fichier séparé, `correspondance_sujet_code.pdf` (7 pages, aussi disponible dans le dossier `reports/` du dépôt). Pour chaque exigence du sujet, il indique le fichier et la fonction qui la réalisent, comment elle est réalisée, et la commande ou le test qui permet de la vérifier. Il n'est pas inclus ici pour respecter la limite de pages. Le tableau ci-dessous donne les commandes qui permettent de refaire chaque résultat du rapport.

Résultat	Commande
Installation	`python -m venv .venv` ; `pip install -e ".[dev]"` ; `npm -prefix ui install`
Interface complète	`python tasks.py demo` (puis `http://127.0.0.1:8000`)
Régénérer les données	`novapolis data`
Validation	`novapolis validate -data data/novapolis`
Module 1	`novapolis access -data data/novapolis -sources casernes`
Module 1, petit jeu	`novapolis minhop -data data/mini -from C2 -to H1`
Module 2 (tab. 1)	`novapolis routes -data data/novapolis -scenario fermeture_axe`
Module 2, D-02 (fig. 4)	`novapolis route -demande D-02 -algo all -scenario fermeture_axe`
Module 3 (tab. 2)	`novapolis negcycle -data data/negatif_accessible -source S -algo all`
Module 4	`novapolis mst -data data/novapolis -algo both`
Module 4, panne	`novapolis mst-panne -data data/novapolis -edge Q-014-Q-019`
Module 5 (tab. 3)	`novapolis scenarios -data data/novapolis -all`
Mesures	`novapolis bench -sizes 250,500,1000,2000,5000 -reps 5 -seed 42`
Figures	`python scripts/make_figures.py reports/bench.csv`
Tests	`pytest`